Rapport de Projet de Fin de Module (PFM)

Avril 1, 2026

Bilal Houari
houari.bilal@etu.uae.ac.ma
Lisence I.D.A.I.

Table des matières

I. Introduction ... 3
I.1. Aperçu du projet ... 3
I.2. Objectifs et buts ... 3
I.3. Aperçu de la pile technologique (Technology Stack) ... 3
II. Conception et mise en œuvre du système ... 4
II.1. Conceptualisation ... 4
II.1.a. Gestion de l'identité des utilisateurs et des rôles ... 4
II.1.b. Entités organisationnelles ... 4
II.1.c. Profils du personnel ... 4
II.1.d. Profils des étudiants ... 4
II.1.e. Coordination académique (Classes et affectations) ... 4
II.1.f. Évaluation et performance ... 5
II.2. Implémentation ... 5
II.2.a. Structure des répertoires ... 5
II.2.b. Architecture de la couche de service ... 6
II.2.b.i. Le Service de base (core/services.py) ... 6
II.2.b.ii. Implémentation de services spécialisés (students/services.py) ... 6
II.2.b.iii. Matrice des responsabilités ... 6
II.2.c. Implémentation Backend et Entités de Données ... 7
II.2.c.i. Identité utilisateur (users/models.py : `CustomUser`) ... 7
II.2.c.ii. Département institutionnel (departments/models.py : `Department`) ... 7
II.2.c.iii. Matière académique (subjects/models.py : `Subject`) ... 8
II.2.c.iv. Personnel éducatif (teachers/models.py : `Teacher`) ... 8
II.2.c.v. Classe scolaire (classes/models.py : `SchoolClass`) ... 9
II.2.c.vi. Profils des étudiants (students/models.py : `Student`) ... 9
II.2.c.vii. Affectations de classe (classes/models.py : `ClassSubjectAssignment`) ... 10
II.2.c.viii. Planification des examens (exams/models.py : `Exam`) ... 10
II.2.c.ix. Résultats académiques (exams/models.py : `ExamResult`) ... 11
II.2.c.x. Calendrier institutionnel (holidays/models.py : `Holiday`) ... 11
II.2.d. Structure du routage des URL ... 12
II.2.d.i. Routage spécifique à l'application ... 12
II.2.d.ii. Conventions de nommage des URL ... 13
II.2.e. Peuplement des données et automatisation ... 13
II.2.f. Hiérarchie des templates et interface utilisateur ... 13
II.2.f.i. Structure des templates de base ... 13
II.2.f.ii. Consultation des pages spécifiques ... 13
II.2.f.iii. Stratégie de style (Styling Strategy) ... 14
III. Fonctionnalités et caractéristiques clés ... 14
III.1. Data Seeder ... 14
III.2. Structure académique et gestion des classes ... 14
III.3. Dashboards basés sur les rôles ... 14
III.4. Gestion du personnel et des programmes ... 15
III.5. Évaluation et suivi des performances ... 15
III.6. Calendrier institutionnel ... 15
IV. Défis et solutions ... 15
IV.1. Consolidation du périmètre et l'entité SchoolClass ... 15
IV.2. Mise en œuvre du frontend et contraintes de templates ... 15

IV.3. Sécurité et contrôle d'accès (RBAC) 16
IV.4. Contraintes de temps et d'échéances 16
V. Défauts et lacunes 16
V.1. Ensemble limité de fonctionnalités académiques 16
V.2. Lacunes institutionnelles et financières 16
V.3. Limitations techniques et de l'interface utilisateur 16
V.4. Tests et assurance qualité 17
VI. Conclusion 17
VI.1. Résultats du projet 17
VI.2. Apprentissages et décisions clés 17
VI.3. Orientations futures 17
VII. Captures d'écran 18

I. Introduction

I.1. Aperçu du projet

Le **PreSkool Management System** est une application web intégrée développée en tant qu'exigence académique finale (PFM) pour le module « Développement Backend Avancé en Python ». Ce projet représente une application pratique de la logique backend fondamentale et avancée, de la conception de bases de données relationnelles et des principes modernes de développement frontend dans un contexte structuré de gestion scolaire. L'objectif principal du système est de fournir une plateforme centralisée facilitant les processus académiques et administratifs essentiels, garantissant une expérience numérique standardisée tant pour le personnel éducatif que pour les étudiants.

I.2. Objectifs et buts

Le projet visait à établir un système d'information de gestion scolaire intégré capable de gérer les entités de données académiques et administratives fondamentales. Les objectifs opérationnels centraux incluaient la fourniture de capacités de gestion complètes pour les domaines suivants :

- **Gestion des dossiers des étudiants** : Fournir un contrôle administratif complet via des opérations CRUD (Create, Read, Update, Delete) pour les profils des étudiants, englobant l'inscription et la maintenance des données personnelles.
- **Administration du personnel académique** : Développer un module dédié à la gestion des dossiers du corps enseignant, incluant les profils professionnels, les spécialisations par sujet et les affiliations départementales.
- **Organisation curriculaire et départementale** : Établir l'infrastructure administrative nécessaire pour organiser la hiérarchie académique de l'école, spécifiquement à travers la création de départements et leur mise en correspondance ultérieure avec les sujets pertinents.
- **Calendriers des vacances académiques** : Implémenter un système standardisé pour la documentation et le suivi des vacances scolaires et des interruptions programmées au cours de l'année académique.
- **Évaluation et saisie des résultats** : Développer un module pour l'administration des examens scolaires et fournir une interface dédiée au corps enseignant pour enregistrer et surveiller les résultats académiques des étudiants.
- **Sécurité et Role-Based Access Control (RBAC)** : Implémenter un cadre d'authentification structuré avec trois rôles d'utilisateur distincts (Administrator, Teacher et Student), chacun défini par des permissions d'accès spécifiques et des interfaces de dashboard spécialisées.

I.3. Aperçu de la pile technologique (Technology Stack)

L'infrastructure technique du système se compose d'une pile technologique standardisée sélectionnée pour sa stabilité et ses performances :

Composant	Technologie	Description
Backend Framework	**Django**	Utilisé pour la logique server-side, la gestion de session et la manipulation de données relationnelles via son ORM.
Database Engine	**SQLite**	Sélectionné pour sa portabilité et ses performances efficaces au sein de l'environnement de développement.
Frontend Technologies	**HTML5, CSS3, JavaScript** (Vanilla)	Outils standards utilisés pour la cohérence visuelle et la maintenabilité sans dépendances à des bibliothèques externes.

II. Conception et mise en œuvre du système

II.1. Conceptualisation

La conceptualisation du système repose sur un modèle de données relationnel qui organise les opérations scolaires en plusieurs entités fonctionnelles. Chaque entité est définie par ses attributs et ses relations avec les autres composants du système.

II.1.a. Gestion de l'identité des utilisateurs et des rôles

Le système utilise un modèle d'utilisateur centralisé pour l'authentification et l'autorisation.

- **Entité `User` de base** : Cette entité utilise l'adresse e-mail comme identifiant principal (username). Elle contient des champs standards tels que le prénom, le nom et le mot de passe.
- **Attribution des rôles** : Un attribut de rôle distingue les utilisateurs en tant qu'Administrator, Teacher ou Student. Cet attribut est utilisé par le système pour déterminer les permissions et les modules de dashboard disponibles.

II.1.b. Entités organisationnelles

La hiérarchie académique est établie par les entités `Department` et `Subject`.

- **Départements** : Définis par un nom unique et une description. Ils servent d'unités organisationnelles principales pour les enseignants et les sujets.
- **Matières** : Chaque `Subject` est associé à un `Department` spécifique (relation Many-to-One). Les sujets contiennent un code unique, généré automatiquement à l'aide d'un préfixe dérivé du nom du département et d'un suffixe aléatoire de trois chiffres.

II.1.c. Profils du personnel

Le système maintient des dossiers détaillés pour le personnel éducatif via l'entité `Teacher`.

- **Liaison avec l'utilisateur** : Chaque dossier d'enseignant est lié à un compte `User` unique par une relation One-to-One.
- **Identification et attributs** : Un identifiant unique (staff ID, format : STAFF-XXX) est attribué aux enseignants. Les attributs incluent la date de naissance, la spécialisation, la date d'embauche, l'adresse et le numéro de téléphone.
- **Association départementale** : Les enseignants sont liés à un `Subject` spécifique, ce qui les associe par la suite à un `Department`.

II.1.d. Profils des étudiants

Les données des étudiants sont gérées via l'entité `Student`, qui suit la progression et le placement individuels.

- **Liaison avec l'utilisateur** : À l'instar des enseignants, chaque dossier d'étudiant est lié à un compte `User` unique via une relation One-to-One.
- **Identification et attributs** : Un numéro d'admission (admission number) unique est attribué aux étudiants, généré automatiquement selon le format ADM-YYYY-XXX. Les attributs incluent la date de naissance, la date d'inscription, l'adresse et le numéro de téléphone.
- **Placement en classe** : Chaque étudiant est affecté à une `SchoolClass` spécifique via une relation Foreign Key.

II.1.e. Coordination académique (Classes et affectations)

Les classes et leurs programmes associés sont gérés via les entités `SchoolClass` et `Assignment`.

- **`SchoolClass`** : Définie par un nom unique et un niveau (grade level, allant de 1 à 12). Elle sert de groupement principal pour les étudiants.
- **`ClassSubjectAssignment`** : Cette entité sert de jonction entre `SchoolClass`, `Subject` et `Teacher`. Elle définit quel enseignant est responsable d'un sujet spécifique au sein d'une classe particulière. Une contrainte d'unicité (unique constraint) garantit qu'un sujet ne peut pas être attribué à plus d'un enseignant par classe.

II.1.f. Évaluation et performance

Les évaluations sont suivies via les entités `Exam` et `ExamResults`.

- **`Exams`** : Définis par un nom, une date et une note maximale (maximum marks). Chaque examen est lié à un `Subject` et une `SchoolClass` spécifiques.
- **`ExamResults`** : Cette entité enregistre la performance d'un `Student` dans un `Exam` spécifique. Elle stocke les notes obtenues et calcule le pourcentage basé sur la note maximale de l'examen.

II.2. Implémentation

La mise en œuvre du système **PreSkool** utilise une architecture Django modulaire conçue pour l'extensibilité et une séparation claire des préoccupations (*separation of concerns*). Cette section fournit une analyse détaillée des composants techniques, de la logique backend et de l'organisation frontend.

II.2.a. Structure des répertoires

Le projet suit une structure modulaire où chaque application représente un domaine fonctionnel distinct. L'arborescence simplifiée ci-dessous met en évidence les composants architecturaux clés, y compris la couche de service (*service layer*) et les outils de gestion.

```text
.
├── admins/
│   └── services.py
├── classes/
│   ├── models.py
│   ├── services.py
│   └── urls.py
├── core/
│   ├── management/commands/seed_data.py
│   ├── mixins.py
│   └── services.py
├── departments/
│   ├── models.py
│   ├── services.py
│   └── urls.py
├── exams/
│   ├── models.py
│   ├── services.py
│   └── urls.py
├── holidays/
│   ├── models.py
│   ├── services.py
│   └── urls.py
├── preskool/                # Project Settings & Root URLS
├── static/                  # Global Assets (CSS/Images)
├── students/                # Student Module
│   ├── models.py
│   ├── services.py
│   └── urls.py
├── subjects/                # Curriculum Module
├── teachers/                # Personnel Module
├── templates/               # UI Layer (Hierarchical)
│   ├── classes/
│   ├── registration/
│   ├── students/
│   └── (base.html, sidebar.html, topbar.html)
└── users/                   # Authentication and RBAC
```

II.2.b. Architecture de la couche de service

Une décision architecturale clé dans ce projet est l'implémentation d'une **Service Layer**. Ce modèle abstrait la logique métier (*business logic*) en dehors des vues, garantissant que les contrôleurs restent légers (*thin controllers*) et que la logique soit réutilisable et testable.

II.2.b.i. Le Service de base (core/services.py)

Tous les services d'application héritent d'un `BaseService`, qui fournit des méthodes standardisées pour les opérations de données de base. Cela garantit que chaque module suit une implémentation cohérente pour les interactions courantes avec la base de données.

```python
class BaseService:
    model = None

    @classmethod
    def get_all(cls):
        """Returns all records for the associated model."""
        return cls.model.objects.all()

    @classmethod
    def get_by_id(cls, obj_id):
        """Retrieves a single record or returns a 404 error."""
        return get_object_or_404(cls.model, id=obj_id)
```

II.2.b.ii. Implémentation de services spécialisés (students/services.py)

Le `StudentService` fournit un exemple pratique de la manière dont la couche de service encapsule une logique métier complexe, telle que la génération de numéros d'admission et les affectations de classes.

```python
class StudentService(BaseService):
    model = Student

    @classmethod
    def create_student(cls, user_data, student_data):
        """Handles the dual creation of a User account and a Student profile."""
        with transaction.atomic():
            user = User.objects.create_user(**user_data)
            student = cls.model.objects.create(user=user, **student_data)
        return student

    @classmethod
    def get_class_mates(cls, student_id):
        """Retrieves all students sharing the same class as the specified student."""
        student = cls.get_by_id(student_id)
        if student.school_class:
            return cls.model.objects.filter(school_class=student.school_class)
        return cls.model.objects.none()
```

II.2.b.iii. Matrice des responsabilités

Le tableau suivant présente la répartition de la logique métier entre les différents services :

Service	Exemple de responsabilité	Implémentation technique
StudentService	Gestion des inscriptions	Gère les transactions atomiques lors de la création d'utilisateur/étudiant.
TeacherService	Coordination du personnel	Valide les associations de départements via les liaisons de sujets.
ClassService	Cartographie académique	Gère la table de jonction `ClassSubjectAssignment`.
ExamService	Traitement des résultats	Calcule les pourcentages des étudiants en utilisant une précision décimale.
HolidayService	Validation du calendrier	Implémente la logique pour vérifier si une date spécifique tombe pendant une pause.

II.2.c. Implémentation Backend et Entités de Données

L'implémentation backend utilise les modèles **Django** pour définir la structure de données relationnelle du système **PreSkool**. Chaque entité représente un domaine conceptuel spécifique, garantissant que les données sont organisées, validées et gérées par programmation.

II.2.c.i. Identité utilisateur (users/models.py : `CustomUser`)

L'entité `CustomUser` est le modèle d'identité fondamental de l'application. En étendant l'**AbstractUser** de Django, le système maintient la compatibilité avec le cadre d'authentification hérité tout en introduisant une logique spécialisée basée sur les rôles.

- **Objectif fonctionnel** : Elle sert de compte primaire pour tous les individus (Admins, Teachers, Students), gérant l'authentification et l'attribution des niveaux de permission.
- **Contexte relationnel** : Elle agit comme l'entité « Parent » pour les profils `Student` et `Teacher` via des relations **One-to-One**.
- **Implémentation** :

```python
class CustomUser(AbstractUser):
    ROLE_CHOICES = (
        ('ADMIN', 'Admin'),
        ('TEACHER', 'Teacher'),
        ('STUDENT', 'Student'),
    )
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='STUDENT')

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']

    def save(self, *args, **kwargs):
        if not self.username:
            self.username = self.email
        super().save(*args, **kwargs)
```

II.2.c.ii. Département institutionnel (departments/models.py : `Department`)

L'entité `Department` organise l'école en unités administratives logiques.

- **Objectif fonctionnel** : Catégoriser les matières académiques et le personnel enseignant dans des domaines d'étude spécifiques (ex: Mathématiques, Sciences).
- **Contexte relationnel** : Elle possède une relation **One-to-Many** avec `Subject` et une relation indirecte avec `Teacher` via leurs matières assignées.
- **Implémentation** :

```python
class Department(models.Model):
    name = models.CharField(max_length=100, unique=True)
    description = models.TextField(blank=True)

    def __str__(self):
        return f"{self.name}"
```

II.2.c.iii. Matière académique (subjects/models.py : `Subject`)

L'entité `Subject` définit les cours spécifiques proposés dans le programme.

- **Objectif fonctionnel** : Représenter les cours individuels et gérer leur catégorisation départementale ainsi que leur identification.
- **Contexte relationnel** : Elle appartient à un `Department` (**Foreign Key**) et est liée aux profils `Teacher` en tant que domaine d'expertise principal. Elle apparaît également dans `ClassSubjectAssignment` pour lier les matières à des classes spécifiques.
- **Implémentation** :

```python
class Subject(models.Model):
    name = models.CharField(max_length=100)
    code = models.CharField(max_length=20, unique=True, blank=True)
    department = models.ForeignKey(Department, on_delete=models.CASCADE, related_name='subjects')

    def save(self, *args, **kwargs):
        if not self.code:
            dept_prefix = self.department.name[:3].upper()
            random_suffix = ''.join(random.choices(string.digits, k=3))
            self.code = f"{dept_prefix}-{random_suffix}"
            while Subject.objects.filter(code=self.code).exists():
                random_suffix = ''.join(random.choices(string.digits, k=3))
                self.code = f"{dept_prefix}-{random_suffix}"
        super().save(*args, **kwargs)
```

II.2.c.iv. Personnel éducatif (teachers/models.py : `Teacher`)

L'entité `Teacher` intègre un profil spécialisé lié à l'identité utilisateur de base pour les données spécifiques au corps enseignant.

- **Objectif fonctionnel** : Gérer les détails du personnel professionnel, incluant l'expertise, l'ancienneté et les marqueurs d'identité.
- **Contexte relationnel** : Liée en **1:1** avec `CustomUser`. Elle possède une relation **Many-to-One** avec `Subject` et apparaît dans plusieurs entrées de `ClassSubjectAssignment`.
- **Implémentation** :

```python
class Teacher(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='teacher_profile')
    staff_id = models.CharField(max_length=20, unique=True, blank=True)
    subject = models.ForeignKey('subjects.Subject', on_delete=models.SET_NULL, null=True, related_name='teachers')
    date_of_birth = models.DateField()
    specialization = models.CharField(max_length=100)
    joining_date = models.DateField(auto_now_add=True)
    address = models.TextField(blank=True)
    phone_number = models.CharField(max_length=15, blank=True)

    def save(self, *args, **kwargs):
        if not self.staff_id:
            random_suffix = ''.join(random.choices(string.digits, k=3))
            self.staff_id = f"STAFF-{random_suffix}"
            while Teacher.objects.filter(staff_id=self.staff_id).exists():
                random_suffix = ''.join(random.choices(string.digits, k=3))
                self.staff_id = f"STAFF-{random_suffix}"
        super().save(*args, **kwargs)
```

II.2.c.v. Classe scolaire (classes/models.py : `SchoolClass`)

L'entité `SchoolClass` facilite le regroupement des étudiants pour l'enseignement académique.

- **Objectif fonctionnel** : Définir les niveaux scolaires et les cohortes administratives au sein de l'institution.
- **Contexte relationnel** : Elle possède une relation **One-to-Many** avec `Student` et sert de clé primaire dans les mappages `ClassSubjectAssignment` et `Exam`.
- **Implémentation** :

```python
class SchoolClass(models.Model):
    name = models.CharField(max_length=50, unique=True)
    grade_level = models.IntegerField() # 1-12
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = "Class"
        verbose_name_plural = "Classes"
        ordering = ['grade_level', 'name']

    def __str__(self):
        return self.name
```

II.2.c.vi. Profils des étudiants (students/models.py : `Student`)

L'entité `Student` représente le profil académique de chaque individu inscrit.

- **Objectif fonctionnel** : Enregistrer les métadonnées spécifiques à l'étudiant et suivre l'affectation en classe.
- **Contexte relationnel** : Liée en **1:1** avec `CustomUser` et appartient à une `SchoolClass` (**Many-to-One**).
- **Implémentation** :

```python
class Student(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='student_profile')
    admission_number = models.CharField(max_length=20, unique=True, blank=True)
    date_of_birth = models.DateField()
    school_class = models.ForeignKey('classes.SchoolClass', on_delete=models.SET_NULL, null=True, related_name='students')
    enrollment_date = models.DateField(auto_now_add=True)
    address = models.TextField(blank=True)
    phone_number = models.CharField(max_length=15, blank=True)

    def save(self, *args, **kwargs):
        if not self.admission_number:
            year = datetime.now().year
            random_suffix = ''.join(random.choices(string.digits, k=3))
            self.admission_number = f"ADM-{year}-{random_suffix}"
            while Student.objects.filter(admission_number=self.admission_number).exists():
                random_suffix = ''.join(random.choices(string.digits, k=3))
                self.admission_number = f"ADM-{year}-{random_suffix}"
        super().save(*args, **kwargs)
```

II.2.c.vii. Affectations de classe (classes/models.py : `ClassSubjectAssignment`)

L'entité `ClassSubjectAssignment` agit comme la jonction primaire pour l'emploi du temps académique.

- **Objectif fonctionnel** : Coordonner la prestation de matières spécifiques à des classes spécifiques en assignant du personnel enseignant dédié.
- **Contexte relationnel** : Lie `SchoolClass`, `Subject` et `Teacher` via trois **Foreign Keys**. Elle impose une contrainte d'unicité sur la combinaison de la classe et de la matière.
- **Implémentation** :

```python
class ClassSubjectAssignment(models.Model):
    school_class = models.ForeignKey(SchoolClass, on_delete=models.CASCADE, related_name='assignments')
    subject = models.ForeignKey(Subject, on_delete=models.CASCADE, related_name='class_assignments')
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE, related_name='class_assignments')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('school_class', 'subject')
        verbose_name = "Class Subject Assignment"
        verbose_name_plural = "Class Subject Assignments"

    def __str__(self):
        return f"{self.school_class.name} - {self.subject.name} ({self.teacher.user.get_full_name()})"
```

II.2.c.viii. Planification des examens (exams/models.py : `Exam`)

L'entité `Exam` facilite la programmation des évaluations des étudiants.

- **Objectif fonctionnel** : Définir les paramètres des évaluations académiques, incluant le calendrier et les plafonds de notation.
- **Contexte relationnel** : Liée à `Subject` et `SchoolClass` (**Foreign Keys**). Elle sert d'entité parente pour plusieurs enregistrements `ExamResult`.
- **Implémentation** :

```python
class Exam(models.Model):
    name = models.CharField(max_length=100)
    date = models.DateField()
    subject = models.ForeignKey(Subject, on_delete=models.CASCADE, related_name='exams')
    school_class = models.ForeignKey('classes.SchoolClass', on_delete=models.CASCADE,
    related_name='exams', null=True)
    max_marks = models.IntegerField(default=100)

    def __str__(self):
        return f"{self.name} - {self.subject.name}"
```

II.2.c.ix. Résultats académiques (exams/models.py : `ExamResult`)

L'entité `ExamResult` capture la performance des étudiants lors des évaluations.

- **Objectif fonctionnel** : Enregistrer les scores numériques et calculer les pourcentages de performance relative pour chaque étudiant.
- **Contexte relationnel** : Liée à un `Exam` et un `Student` spécifiques (**Foreign Keys**).
- **Implémentation** :

```python
class ExamResult(models.Model):
    exam = models.ForeignKey(Exam, on_delete=models.CASCADE, related_name='results')
    student = models.ForeignKey(Student, on_delete=models.CASCADE, related_name='exam_results')
    marks_obtained = models.DecimalField(max_digits=5, decimal_places=2)

    def __str__(self):
        return f"{self.student.user.get_full_name()} - {self.exam.name}: {self.marks_obtained}"

    @property
    def percentage(self):
        if self.exam.max_marks > 0:
            return (self.marks_obtained / self.exam.max_marks) * 100
        return 0
```

II.2.c.x. Calendrier institutionnel (holidays/models.py : `Holiday`)

L'entité `Holiday` maintient le calendrier institutionnel pour les périodes non académiques.

- **Objectif fonctionnel** : Documenter les interruptions et les vacances à l'échelle de l'école.
- **Contexte relationnel** : Il s'agit d'une entité autonome qui fournit un contexte temporel à l'année académique de l'école.
- **Implémentation** :

```python
class Holiday(models.Model):
    name = models.CharField(max_length=100)
    start_date = models.DateField()
    end_date = models.DateField()
    description = models.TextField(blank=True)

    def __str__(self):
        return f"{self.name} ({self.start_date})"

    @property
    def duration(self):
        diff = self.end_date - self.start_date
        return diff.days + 1
```

II.2.d. Structure du routage des URL

L'architecture de routage du système **PreSkool** est implémentée à l'aide d'une stratégie décentralisée. Cela implique une configuration à deux niveaux où le projet racine gère les préfixes de domaine de haut niveau et les applications individuelles gèrent leur propre logique de point de terminaison (**endpoint**) localisée.

II.2.d.i. Routage spécifique à l'application

Chaque application maintient un fichier `urls.py` interne, qui mappe des **patterns** localisés à des vues spécifiques. Cette modularité garantit que la base de code reste organisée et que les modifications apportées à un module n'impactent pas les autres par inadvertance.

1. Dossiers étudiants et académiques (students/urls.py) Ce routeur gère toutes les interactions spécifiques aux étudiants, allant des vues de **dashboard** à la gestion administrative.

Pattern	View Class	Description
`''`	`StudentListView`	Liste globale des étudiants inscrits.
`'dashboard/'`	`StudentDashboardView`	Aperçu personnalisé de l'étudiant (Résultats, Camarades de classe).
`'add/'`	`StudentCreateView`	Formulaire administratif pour l'inscription des étudiants.
`'edit/<int:pk>/'`	`StudentUpdateView`	Modification de profil et maintenance des données.
`'my-class/'`	`MyClassView`	Vue de la cohorte de classe assignée à l'étudiant.

2. Coordination des classes et du programme (classes/urls.py)
Le routeur `classes` gère la mise en correspondance des niveaux scolaires et les affectations du corps enseignant à des matières spécifiques.

Pattern	View Class	Description
`''`	`ClassListView`	Index de tous les niveaux scolaires actifs.
`'<int:pk>/'`	`ClassDetailView`	Analyse approfondie des étudiants d'une classe et des affectations d'enseignants.
`'assignment/add/'`	`AssignmentCreateView`	Formulaire spécialisé pour l'affectation d'un enseignant et d'une matière à une classe.

3. Évaluation et examen (exams/urls.py)
Le routage pour le module d'évaluation est conçu pour gérer à la fois la planification et la saisie de données à haut volume requise pour la notation.

Pattern	View Class	Description
`''`	`ExamListView`	Liste chronologique des examens programmés.
`'add/'`	`ExamCreateView`	Interface pour la planification de nouvelles évaluations.
`'results/entry/<int:exam_id>/'`	`ExamResultEntryView`	Portail spécialisé de saisie de données en masse pour la notation par les enseignants.

4. Programmation institutionnelle et entités (holidays/urls.py)
Les applications administratives secondaires (Holidays, Departments, Subjects) suivent un **pattern** de routage **CRUD** standardisé, garantissant la cohérence des conventions de nommage des points de terminaison.

- **Structure de pattern exemple** :

- `list/` : Pour les aperçus de données.
- `add/` : Pour la création de nouveaux enregistrements.
- `edit/<int:pk>/` : Pour la modification d'enregistrements.
- `delete/<int:pk>/` : Pour la suppression d'enregistrements.

II.2.d.ii. Conventions de nommage des URL

Le projet utilise une convention de nommage cohérente (`app-action`) pour chaque route. Cela permet d'obtenir des liens de référence croisée propres, lisibles et maintenables dans toute la couche de **template** en utilisant la balise `{% url %}`.

II.2.e. Peuplement des données et automatisation

Le projet inclut un **data seeder** complet situé dans `core/management/commands/seed_data.py`. Cette commande constitue l'une des parties les plus critiques de l'infrastructure de l'application, car elle automatise la création d'un environnement de test complet et réaliste.

Capacités du Seeder :

1. **Génération d'identités (Identity Generation)** : Le **seeder** crée un ensemble standard de comptes de test avec des mots de passe fixes (hachage `pbkdf2_sha256`), garantissant que les développeurs peuvent se connecter immédiatement en tant qu'**Admin**, **Teacher** ou **Student**.
2. **Complexité relationnelle** : Il construit par programmation la hiérarchie académique, en créant d'abord les entités `Department`, suivies des enregistrements `Subject` et `SchoolClass`, garantissant que toutes les contraintes de clés étrangères (**foreign key**) sont respectées.
3. **Peuplement réaliste des données** : À l'aide de boucles et de logiques conditionnelles, il peuple la base de données avec des nombres réalistes d'inscriptions d'étudiants, assigne plusieurs matières aux classes et lie les enseignants en fonction de leurs matières de spécialisation.
4. **Historique académique** : Il génère des événements d'examens (`Exam`) historiques et les enregistrements `ExamResult` correspondants, fournissant des données immédiates pour les **dashboards** de performance.

```python
# core/management/commands/seed_data.py snippet
def handle(self, *args, **options):
    self.stdout.write('Seeding database...')
    # 1. Create Departments
    science = Department.objects.create(name='Science')
    # 2. Create Subjects
    maths = Subject.objects.create(name='Mathematics', department=science)
    # 3. Create Classes
    class_10 = SchoolClass.objects.create(name='Grade 10A', grade_level=10)
    # ... logic continues for Teachers and Students
```

II.2.f. Hiérarchie des templates et interface utilisateur

La couche **UI** est organisée en une structure hiérarchique qui maximise la réutilisation du code via l'héritage de **templates** et les **partials**.

II.2.f.i. Structure des templates de base

- **`base.html`** : Le layout principal contenant la structure du document HTML et les injections globales de scripts et de styles.
- **`topbar.html`** : Contient le menu du profil utilisateur et les notifications spécifiques aux rôles.
- **`sidebar.html`** : Implémente le menu de navigation rétractable, avec une visibilité des éléments contrôlée par le rôle de l'utilisateur.

II.2.f.ii. Consultation des pages spécifiques

Les pages sont regroupées en blocs fonctionnels :

- **Dashboards de gestion** : `student_dashboard.html`, `teacher_dashboard.html`, `home.html` (**dashboard Admin**).

- **Interfaces CRUD** : Formulaires standardisés (`student_form.html`, `exam_form.html`) pour la création et l'édition d'enregistrements.
- **Vues académiques** : `my_class.html` (vue étudiant), `teacher_class_list.html` (vue enseignant) et `exam_result_entry.html`.
- **Listes administratives** : Listes pour les départements, les matières, les vacances et les classes.

II.2.f.iii. Stratégie de style (Styling Strategy)

Le style est strictement géré à l'aide de **vanilla CSS**, réparti sur trois fichiers pour maintenir une séparation claire des préoccupations (**Layout**, **Components** et **Global Variables**).

III. Fonctionnalités et caractéristiques clés

Le **PreSkool Management System** intègre plusieurs modules pour traiter les données administratives et académiques. Chaque fonctionnalité est conçue pour automatiser les flux de travail institutionnels et maintenir la cohérence des données sur l'ensemble de la plateforme.

III.1. Data Seeder

Le **Data Seeder** (`seed_data.py`) est un composant administratif central utilisé pour initialiser la base de données du système. Il est conçu à la fois pour les tests de développement et pour les démonstrations du système.

- **Configuration institutionnelle** : Le **seeder** génère la hiérarchie scolaire, incluant les départements, les matières et les niveaux scolaires, en utilisant une seule commande de gestion.
- **Génération de comptes utilisateurs** : Il crée un ensemble standardisé de comptes utilisateurs (**Admin**, **Teacher** et **Student**) avec des identifiants prédéfinis.
- **Mappage relationnel** : Le **seeder** implémente des mappages relationnels entre les enseignants, les matières et les classes. Cela inclut la création d'**assignments** et la génération de résultats d'examens pour vérifier la logique du système.
- **Peuplement de données standard** : Il peuple la base de données avec des données synchronisées, telles que l'inscription des étudiants dans des classes spécifiques et les dossiers d'examen correspondants.

III.2. Structure académique et gestion des classes

L'entité **School Class** a été implémentée pour fournir une structure logique pour les étudiants et les programmes.

- **Structure organisationnelle** : Bien que non explicitement requise par les spécifications initiales, le modèle `SchoolClass` a été ajouté pour fournir un mécanisme de regroupement pour les étudiants. Cela prévient les problèmes administratifs qui surviennent lors de la gestion d'étudiants sur plusieurs niveaux scolaires sans une hiérarchie basée sur les classes.
- **Hub administratif** : L'entité classe sert de cible pour les **assignments** matières-enseignants et l'inscription des étudiants.
- **Suivi du niveau scolaire** : Les classes sont regroupées par niveau scolaire (1-12), garantissant que la progression académique des étudiants est enregistrée de manière systématique.

III.3. Dashboards basés sur les rôles

Le système fournit trois interfaces de **dashboard** distinctes basées sur le rôle de l'utilisateur.

- **Administrator Dashboard** : Fournit un accès **CRUD** aux entités du système, incluant le personnel, les départements et les calendriers de vacances.
- **Teacher Dashboard** : Affiche les classes et les matières assignées. Les enseignants peuvent accéder au module de saisie des résultats pour leurs examens.

- **Student Dashboard** : Affiche les **assignments** de classe, les vacances scolaires et les résultats d'examens personnels.

III.4. Gestion du personnel et des programmes

La plateforme gère le personnel scolaire et le programme académique à travers des modules spécialisés.

- **Gestion des enseignants** : Enregistre les spécialisations professionnelles et l'expertise par matière pour les associations enseignants-départements.
- **Organisation du programme** : Les matières sont organisées par département, fournissant une structure pour le programme académique.

III.5. Évaluation et suivi des performances

Le module d'évaluation gère la planification et la notation des évaluations des étudiants.

- **Planification des examens** : Permet la planification des examens pour des matières et des classes spécifiques.
- **Enregistrement des résultats** : Stocke les scores numériques et calcule les pourcentages de performance pour chaque étudiant.

III.6. Calendrier institutionnel

- **Gestion des vacances** : Une interface dédiée pour l'enregistrement des vacances à l'échelle de l'école et des interruptions programmées.

IV. Défis et solutions

La mise en œuvre du **PreSkool Management System** a été confrontée à plusieurs défis techniques et logistiques qui ont nécessité des ajustements significatifs de l'architecture du projet et de la stratégie de développement.

IV.1. Consolidation du périmètre et l'entité SchoolClass

Défi : La spécification initiale du projet n'exigeait pas explicitement une entité `SchoolClass` autonome. Au cours de la phase de développement précoce, il est devenu évident que la gestion des étudiants, des matières et des enseignants sans un pôle organisationnel centralisé entraînerait des mappages de données incohérents et un flux de travail administratif désordonné.

Solution : Pour éviter cela, l'entité `SchoolClass` a été introduite comme un composant central du modèle de données. Cela a permis de fournir un regroupement logique pour les étudiants et une cible pour les **assignments** matières-enseignants, garantissant ainsi l'intégrité relationnelle du système.

IV.2. Mise en œuvre du frontend et contraintes de templates

Défi : Un ensemble préexistant de ressources **Bootstrap** avait été initialement fourni pour être utilisé dans le **frontend** du projet. Cependant, ces ressources contenaient une structure de fichiers complexe et redondante, composée de plus de 560 fichiers répartis dans 110 répertoires. Le degré élevé d'imbrication et de duplication rendait difficile l'intégration efficace de ces ressources dans le système d'héritage de **templates Django** dans le délai imparti au projet.

Solution : Afin de garantir une base de code propre et maintenable, la décision a été prise de développer une interface utilisateur personnalisée à partir de zéro. Cette approche a permis de mettre en œuvre une structure de **template** plus efficace et un **static asset pipeline** plus léger. Un relevé complet de la structure originale des fichiers de ressources est documenté dans `/report/template-file-tree.md`.

IV.3. Sécurité et contrôle d'accès (RBAC)

Défi : L'implémentation d'un accès cohérent basé sur les rôles à travers huit applications distinctes nécessitait un mécanisme de sécurité centralisé pour empêcher l'accès non autorisé aux données entre les **Admins**, les **Teachers** et les **Students**.

Solution : Ce problème a été résolu en développant des **mixins** spécialisés dans l'application `core`, tels que `AdminRequiredMixin` et `TeacherRequiredMixin`. Ces **mixins** ont été appliqués à toutes les **class-based views**, garantissant que la logique d'autorisation soit appliquée au niveau de la vue avant tout traitement de données.

IV.4. Contraintes de temps et d'échéances

Le projet était soumis à des limitations temporelles strictes, avec une date limite de soumission finale fixée au 2 avril 2026. La fenêtre de développement était d'environ 10 jours, durant lesquels tous les modules principaux, la **Service Layer** et l'**UI** personnalisée devaient être entièrement implémentés et testés.

V. Défauts et lacunes

Le **PreSkool Management System** a été développé dans un délai restreint de 10 jours pour respecter l'échéance du 2 avril 2026. Par conséquent, l'application se concentre sur les fonctionnalités administratives de base et présente plusieurs lacunes fonctionnelles et techniques qui devraient être corrigées pour un déploiement de niveau production.

V.1. Ensemble limité de fonctionnalités académiques

En raison de la priorité accordée à la gestion de l'identité et des dossiers de base, plusieurs modules académiques avancés n'ont pas été implémentés :

- **Module d'emploi du temps manquant** : Le système ne dispose pas d'un moteur de planification pour gérer les horaires de cours quotidiens, l'attribution des salles et les rotations basées sur les périodes.
- **Absence de gestion des devoirs** : Bien que le système gère les examens formels et les résultats, il ne prend pas actuellement en charge la distribution, la soumission ou la notation des tâches d'évaluation continue (**assignments**).
- **Aucun suivi de l'assiduité** : Les dossiers institutionnels ne comprennent pas de mécanisme pour documenter l'assiduité quotidienne des étudiants ou la présence des enseignants.

V.2. Lacunes institutionnelles et financières

Le périmètre du système a été limité à la gestion académique, laissant d'autres opérations scolaires critiques sans solution :

- **Absence de modules financiers** : Il n'existe aucune fonctionnalité pour la gestion des frais scolaires, de la paie du personnel enseignant ou du suivi des dépenses institutionnelles.
- **Absence de couche de communication** : La plateforme ne facilite pas la communication directe entre les administrateurs, les enseignants et les étudiants (ex : messagerie interne ou systèmes de notification).

V.3. Limitations techniques et de l'interface utilisateur

- **Maturité de l'interface utilisateur** : Bien que l'interface personnalisée en **vanilla CSS** offre une meilleure modularité que le pack de ressources fourni, il s'agit d'une conception de type **Minimum Viable Product (MVP)**. Elle manque de raffinement, d'animations avancées et des fonctionnalités d'accessibilité attendues d'un portail de gestion scolaire de niveau production.
- **Absence de recherche et de filtrage** : L'implémentation actuelle n'inclut aucune fonctionnalité de recherche ou de filtrage. Les utilisateurs doivent identifier manuellement les enregistrements dans les vues de liste

administratives, ce qui constitue une limitation majeure pour les systèmes disposant de grands ensembles de données.
- **Absence de pagination** : Tous les enregistrements sont actuellement chargés sur une seule page sans pagination. Cela nécessite l'utilisation du défilement continu (**mousewheel**) et ne constitue pas une solution évolutive pour un usage institutionnel.
- **Génération de rapports et de documents** : Le système ne permet pas de générer des documents officiels, tels que des bulletins de notes au format PDF, des relevés de notes ou des certificats d'inscription.
- **Gestion des médias statiques** : Le système ne prend pas actuellement en charge le téléchargement de fichiers pour les photos de profil des étudiants/enseignants ou le stockage de documents pour les ressources pédagogiques.
- **Moteur de base de données (SQLite)** : Le projet utilise **SQLite** pour la gestion de la base de données. Bien qu'adapté au développement académique, ce moteur n'est pas conçu pour les accès concurrents et les volumes de données requis par un établissement d'enseignement réel.

V.4. Tests et assurance qualité

- **Suite de tests automatisés** : En raison du cycle de développement rapide, le projet n'inclut pas de suite complète de tests unitaires ou d'intégration automatisés. La vérification a été effectuée manuellement à l'aide du **data seeder** pour simuler des environnements institutionnels.

VI. Conclusion

Le développement du **PreSkool Management System** a abouti à une application backend modulaire et fonctionnelle qui remplit les exigences clés pour la gestion des dossiers à l'échelle de l'école. Malgré le délai strict de 10 jours et les contraintes posées par la structure initiale des ressources, le produit final démontre une implémentation robuste du framework **Django**.

VI.1. Résultats du projet

Le projet a livré avec succès une plateforme administrative multi-rôles qui gère :

- **Identity Management** : Un accès centralisé basé sur les rôles pour les **Admins**, les **Teachers** et les **Students**.
- **Academic Hierarchy** : Une organisation complète des départements, des matières et des niveaux scolaires.
- **Dynamic Assessment** : La planification des examens et l'évaluation automatisée des performances.

VI.2. Apprentissages et décisions clés

La décision d'implémenter une **Service Layer** personnalisée et une entité **SchoolClass** dédiée s'est avérée essentielle pour maintenir une base de code propre et évolutive. De plus, la construction d'une **Vanilla CSS UI** personnalisée a permis un système d'héritage de **templates** plus efficace et un cycle de développement plus rapide qu'il n'aurait été possible avec le pack de ressources fourni.

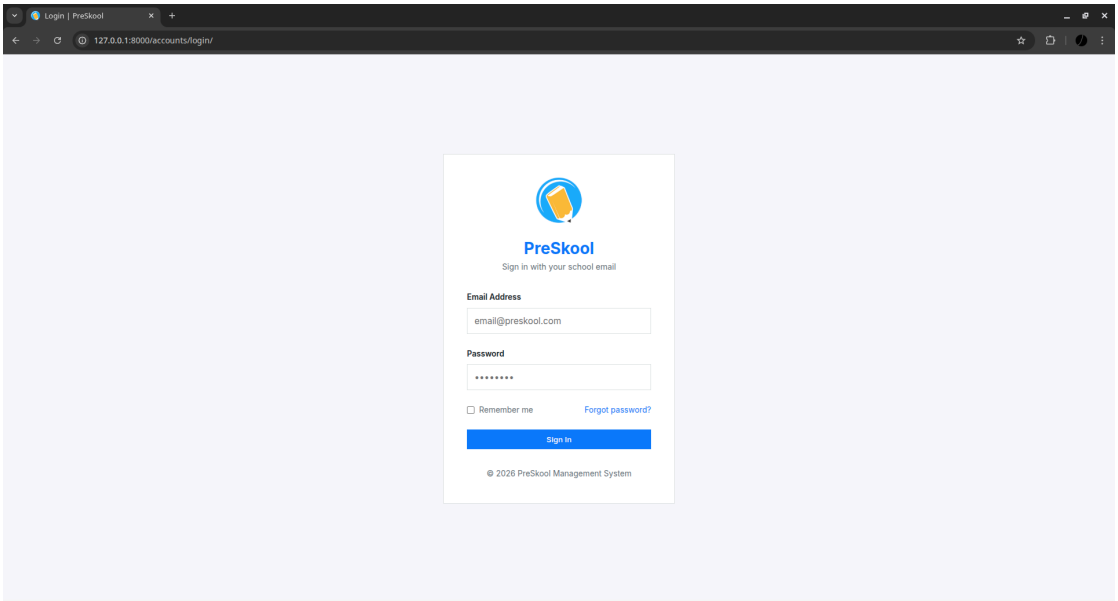
VI.3. Orientations futures

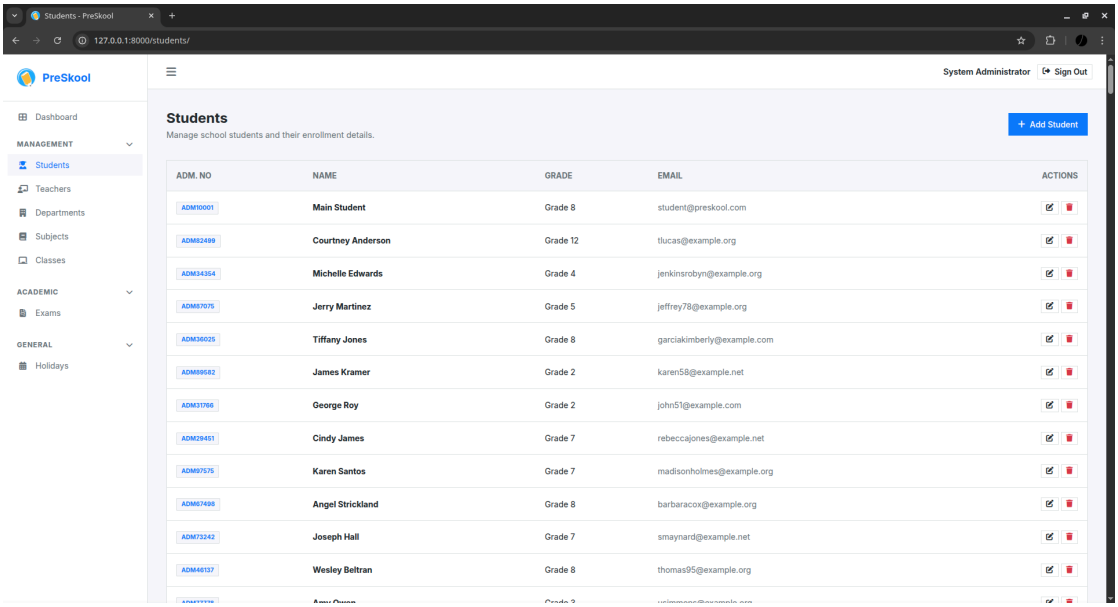
La version actuelle du système est un modèle de démonstration et ne convient pas à un déploiement dans un environnement institutionnel réel. Pour toute tentative future, des changements significatifs dans la pile technologique seraient nécessaires pour garantir des performances et une évolutivité de niveau production :

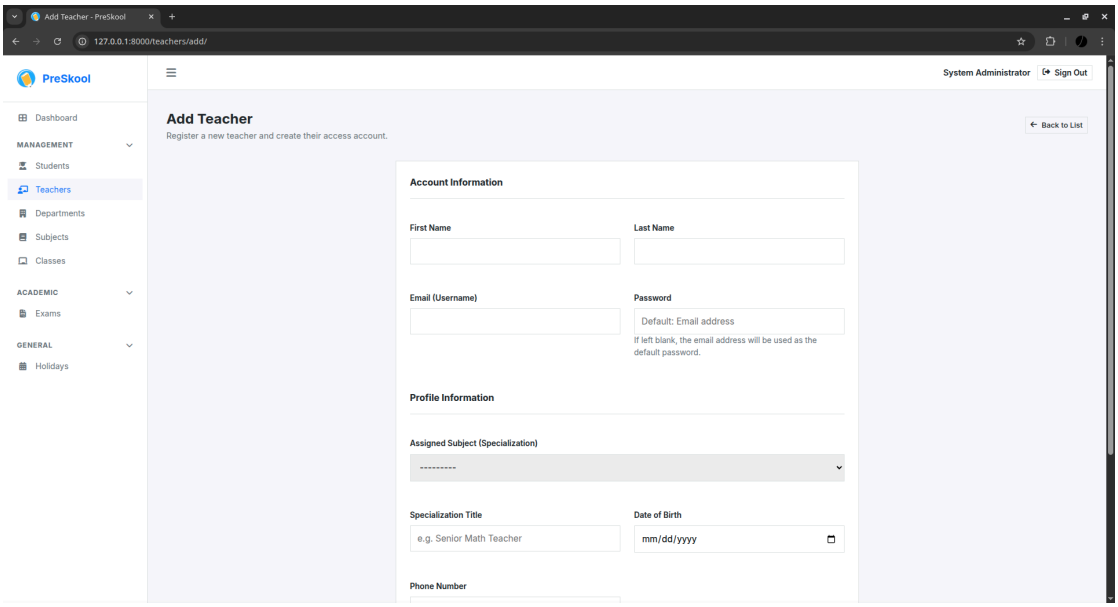
- **Frontend Frameworks** : Intégration d'un framework **frontend** dédié (par exemple, **React** ou **Next.js**) pour gérer l'interface utilisateur indépendamment du backend **Django**.
- **Production-Grade Database** : Transition de **SQLite** vers un système de gestion de base de données relationnelle robuste tel que **PostgreSQL** pour gérer les utilisateurs concurrents et l'intégrité des données à grande échelle.
- **Advanced Features** : Inclusion de la planification en temps réel (**Timetables**), du suivi des transactions financières (**School Fees**) et de la génération automatisée de documents (**Transcripts**).

En conclusion, le système **PreSkool** sert de **Proof of Concept (POC)** fonctionnel pour un système administratif basé sur **Django**, en priorisant le développement modulaire et la structure relationnelle dans un délai limité.

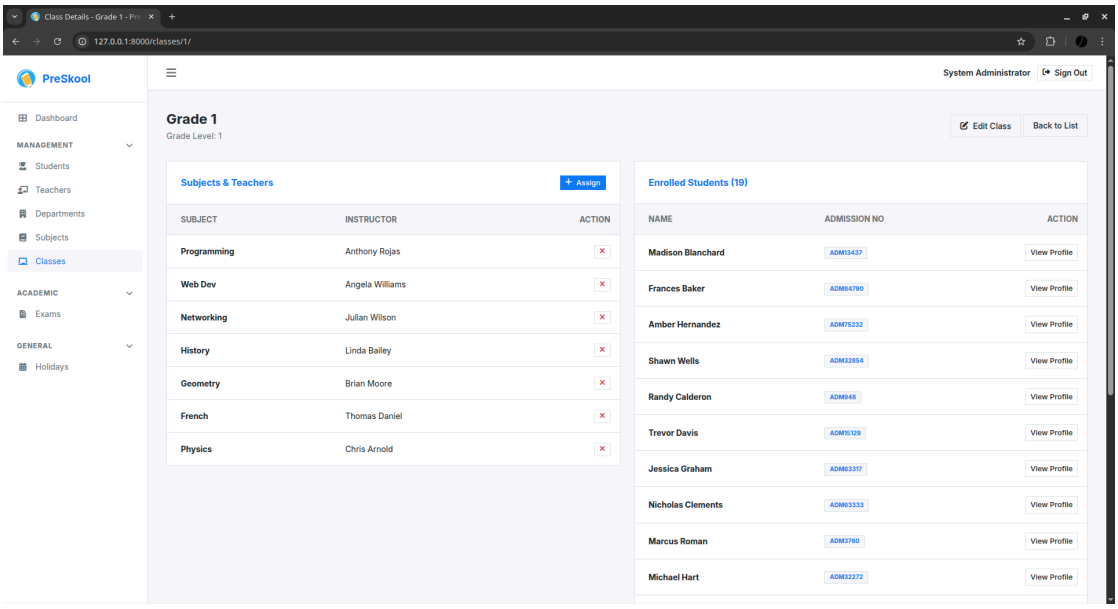
VII. Captures d'écran

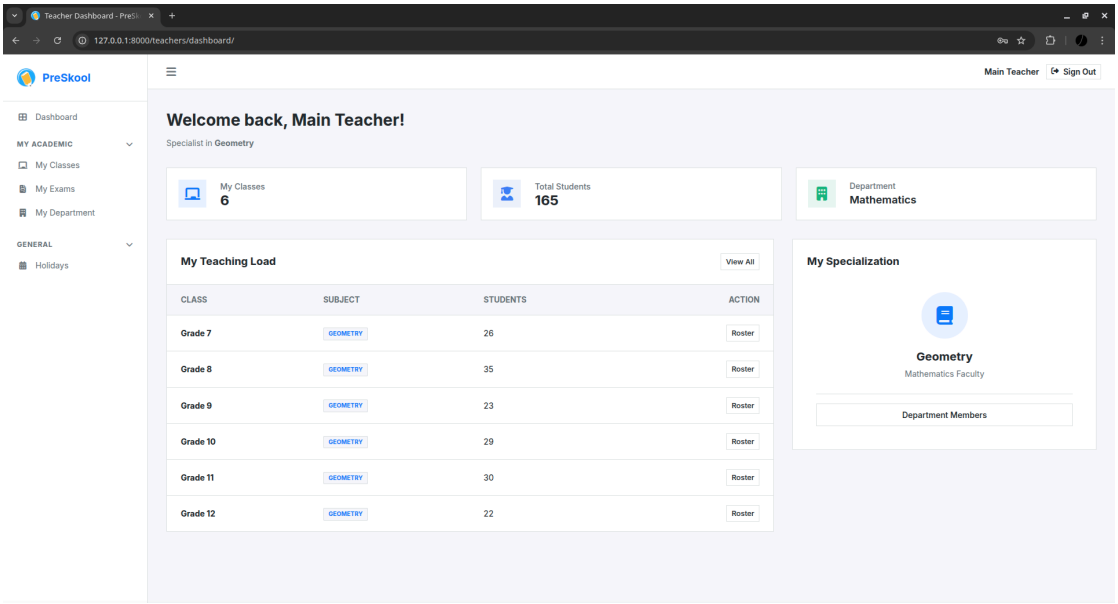


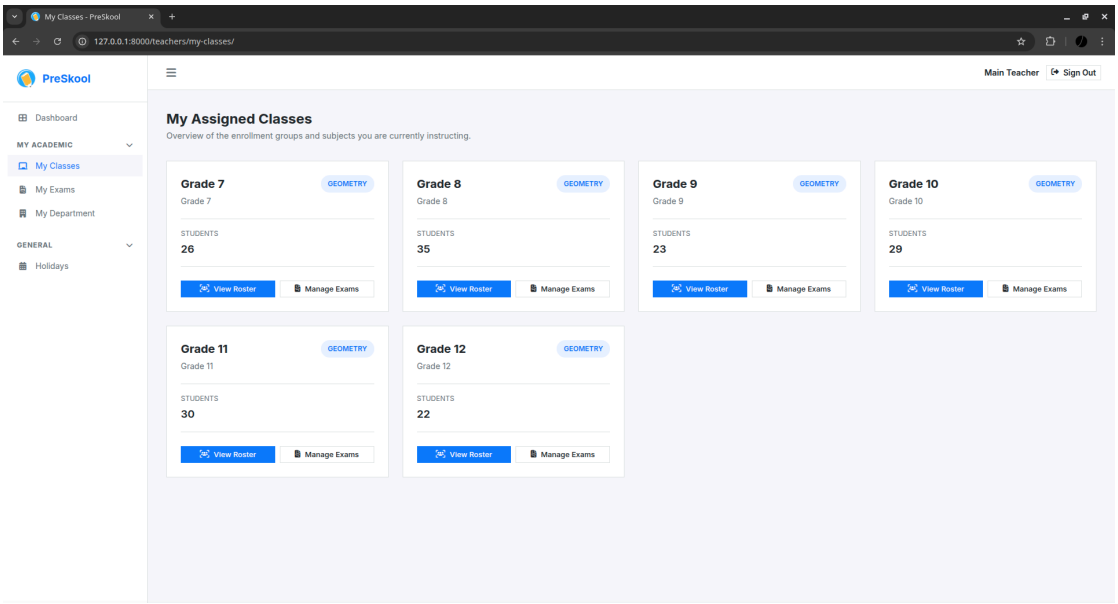


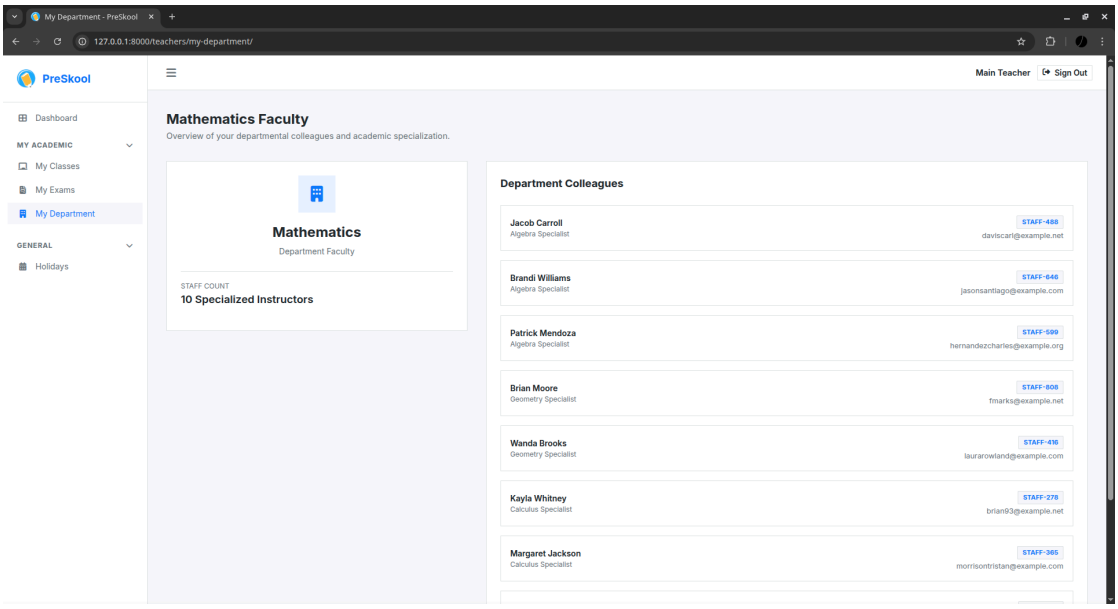

Add Teacher - PreSkool
127.0.0.1:8000/teachers/add/
PreSkool
System Administrator
Sign Out
Dashboard
MANAGEMENT
Students
Teachers
Departments
Subjects
Classes
ACADEMIC
Exams
GENERAL
Holidays
Add Teacher
Register a new teacher and create their access account.
Back to List
Account Information
First Name
Last Name
Email (Username)
Password
Default: Email address
If left blank, the email address will be used as the default password.
Profile Information
Assigned Subject (Specialization)

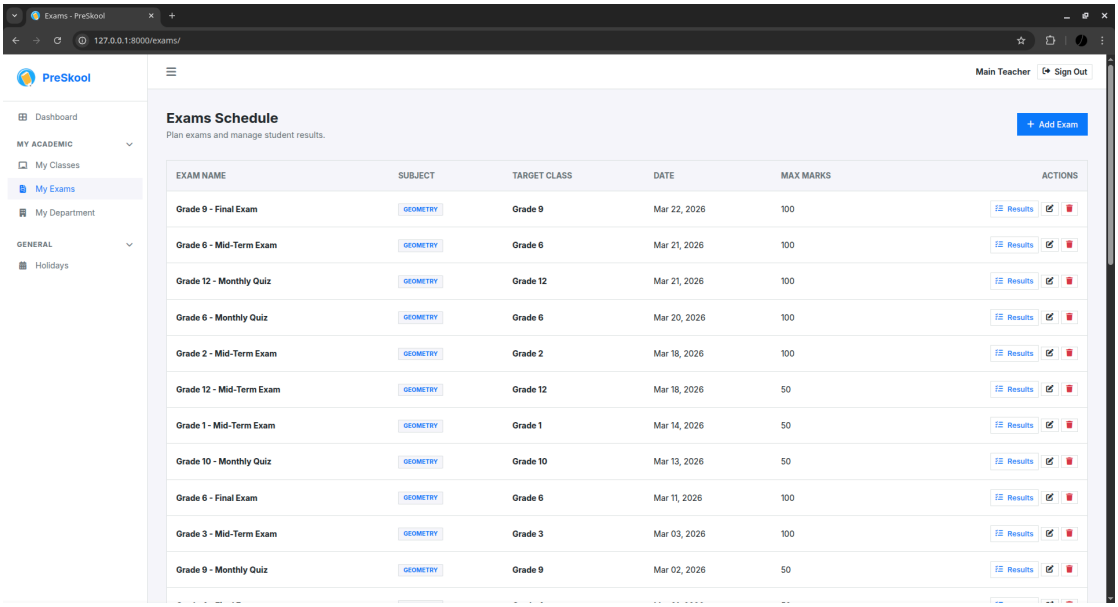
Specialization Title
e.g. Senior Math Teacher
Date of Birth
mm/dd/yyyy
Phone Number

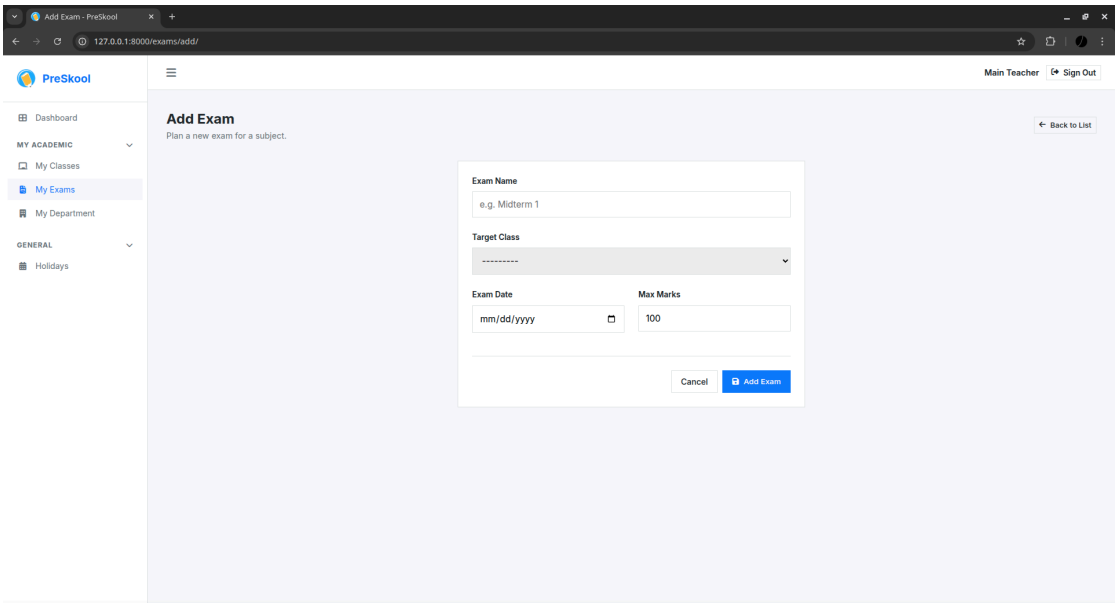

Class Details - Grade 1 - Pre
127.0.0.1:8000/classes/1/
PreSkool
System Administrator
Sign Out
Dashboard
MANAGEMENT
Students
Teachers
Departments
Subjects
Classes
ACADEMIC
Exams
GENERAL
Holidays
Grade 1
Grade Level: 1
Edit Class
Back to List
Subjects & Teachers
Assign
SUBJECT
INSTRUCTOR
ACTION
Programming
Anthony Rojas
Web Dev
Angela Williams
Networking
Julian Wilson
History
Linda Bailey
Geometry
Brian Moore
French
Thomas Daniel
Physics
Chris Arnold
Enrolled Students (19)
NAME
ADMISSION NO
ACTION
Madison Blanchard
ADM13437
View Profile
Frances Baker
ADM84790
View Profile
Amber Hernandez
ADM75232
View Profile
Shawn Wells
ADM32854
View Profile
Randy Calderon
ADM948
View Profile
Trevor Davis
ADM15129
View Profile
Jessica Graham
ADM63317
View Profile
Nicholas Clements
ADM63333
View Profile
Marcus Roman
ADM3760
View Profile
Michael Hart
ADM32272
View Profile


Teacher Dashboard - PreSk
127.0.0.1:8000/teachers/dashboard/
PreSkool
Main Teacher
Sign Out
Dashboard
MY ACADEMIC
My Classes
My Exams
My Department
GENERAL
Holidays
Welcome back, Main Teacher!
Specialist in Geometry
My Classes
6
Total Students
165
Department
Mathematics
My Teaching Load
View All
CLASS
SUBJECT
STUDENTS
ACTION
Grade 7
GEOMETRY
26
Roster
Grade 8
GEOMETRY
35
Roster
Grade 9
GEOMETRY
23
Roster
Grade 10
GEOMETRY
29
Roster
Grade 11
GEOMETRY
30
Roster
Grade 12
GEOMETRY
22
Roster
My Specialization
Geometry
Mathematics Faculty
Department Members

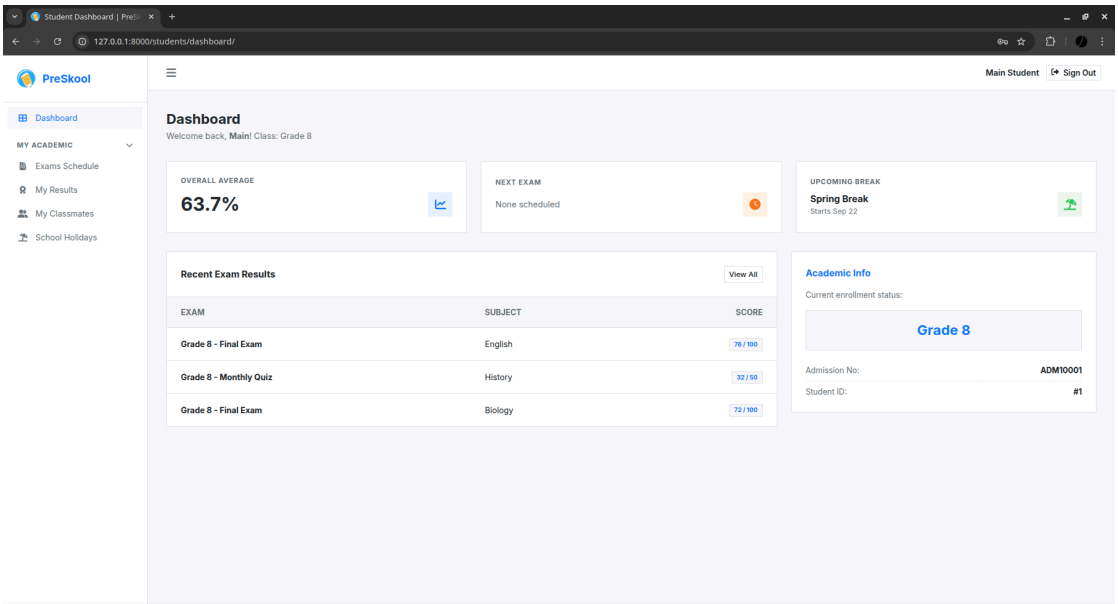

My Classes - PreSkool
127.0.0.1:8000/teachers/my-classes/
PreSkool
Main Teacher
Sign Out
Dashboard
MY ACADEMIC
My Classes
My Exams
My Department
GENERAL
Holidays
My Assigned Classes
Overview of the enrollment groups and subjects you are currently instructing.
Grade 7
GEOMETRY
Grade 7
STUDENTS
26
View Roster
Manage Exams
Grade 8
GEOMETRY
Grade 8
STUDENTS
35
View Roster
Manage Exams
Grade 9
GEOMETRY
Grade 9
STUDENTS
23
View Roster
Manage Exams
Grade 10
GEOMETRY
Grade 10
STUDENTS
29
View Roster
Manage Exams
Grade 11
GEOMETRY
Grade 11
STUDENTS
30
View Roster
Manage Exams
Grade 12
GEOMETRY
Grade 12
STUDENTS
22
View Roster
Manage Exams

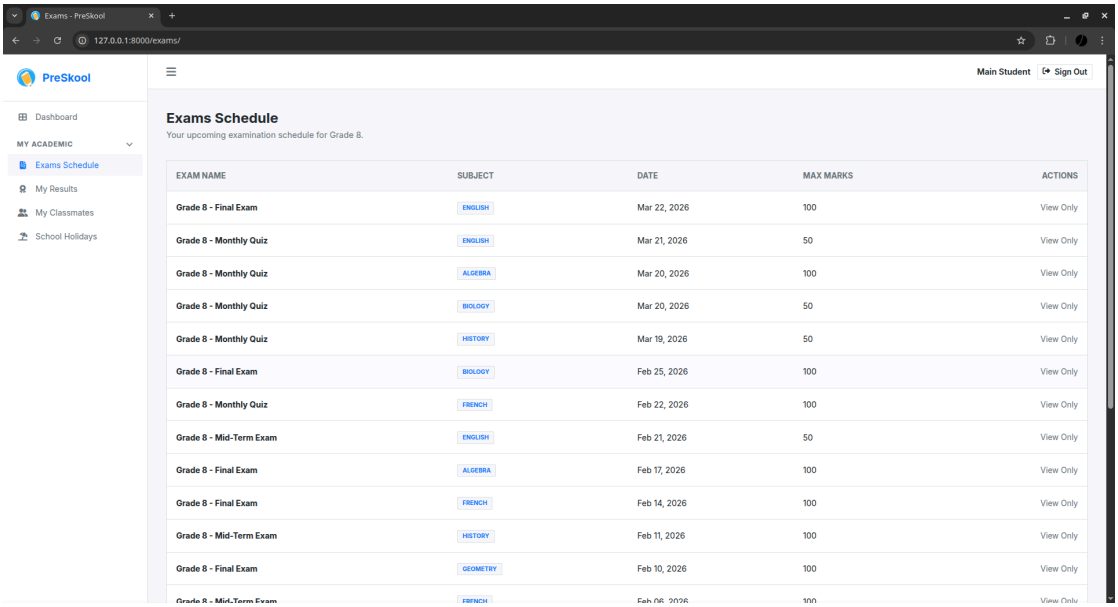

My Department - PreSkool
127.0.0.1:8000/teachers/my-department/
PreSkool
Main Teacher
Sign Out
Dashboard
MY ACADEMIC
My Classes
My Exams
My Department
GENERAL
Holidays
Mathematics Faculty
Overview of your departmental colleagues and academic specialization.
Mathematics
Department Faculty
STAFF COUNT
10 Specialized Instructors
Department Colleagues
Jacob Carroll
Algebra Specialist
STAFF-488
davlscarl@example.net
Brandi Williams
Algebra Specialist
STAFF-646
jasonsantiago@example.com
Patrick Mendoza
Algebra Specialist
STAFF-599
hernandezcharles@example.org
Brian Moore
Geometry Specialist
STAFF-808
fmarks@example.net
Wanda Brooks
Geometry Specialist
STAFF-416
laurarowland@example.com
Kayla Whitney
Calculus Specialist
STAFF-278
brian93@example.net
Margaret Jackson
Calculus Specialist
STAFF-365
morrisontristan@example.com

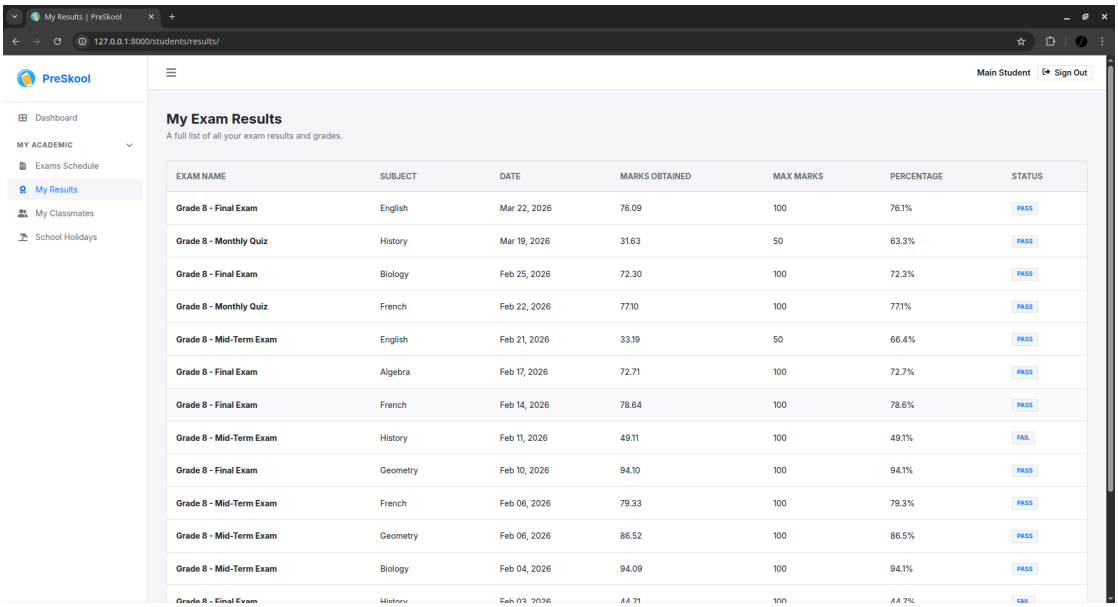

Exams - PreSkool
127.0.0.1:8000/exams/
PreSkool
Main Teacher
Sign Out
Dashboard
MY ACADEMIC
My Classes
My Exams
My Department
GENERAL
Holidays
Exams Schedule
Plan exams and manage student results.
Add Exam
EXAM NAME
SUBJECT
TARGET CLASS
DATE
MAX MARKS
ACTIONS
Grade 9 - Final Exam
GEOMETRY
Grade 9
Mar 22, 2026
100
Grade 6 - Mid-Term Exam
GEOMETRY
Grade 6
Mar 21, 2026
100
Grade 12 - Monthly Quiz
GEOMETRY
Grade 12
Mar 21, 2026
100
Grade 6 - Monthly Quiz
GEOMETRY
Grade 6
Mar 20, 2026
100
Grade 2 - Mid-Term Exam
GEOMETRY
Grade 2
Mar 18, 2026
100
Grade 12 - Mid-Term Exam
GEOMETRY
Grade 12
Mar 18, 2026
50
Grade 1 - Mid-Term Exam
GEOMETRY
Grade 1
Mar 14, 2026
50
Grade 10 - Monthly Quiz
GEOMETRY
Grade 10
Mar 13, 2026
50
Grade 6 - Final Exam
GEOMETRY
Grade 6
Mar 11, 2026
100
Grade 3 - Mid-Term Exam
GEOMETRY
Grade 3
Mar 03, 2026
100
Grade 9 - Monthly Quiz
GEOMETRY
Grade 9
Mar 02, 2026
50
Results

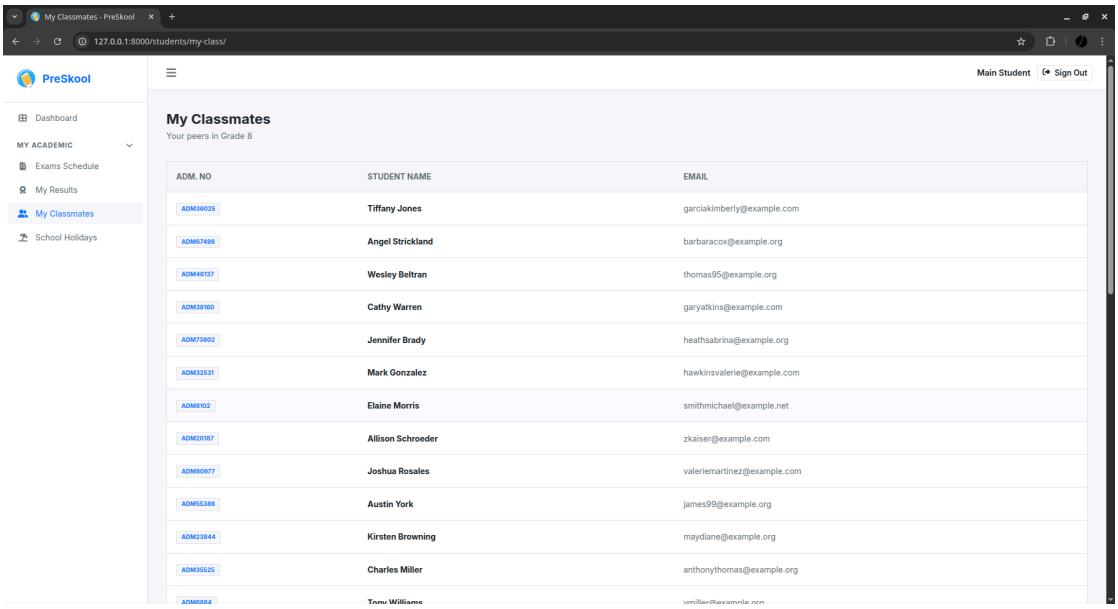

Add Exam - PreSkool
127.0.0.1:8000/exams/add/
PreSkool
Main Teacher
Sign Out
Dashboard
MY ACADEMIC
My Classes
My Exams
My Department
GENERAL
Holidays
Add Exam
Plan a new exam for a subject.
Back to List
Exam Name
e.g. Midterm 1
Target Class

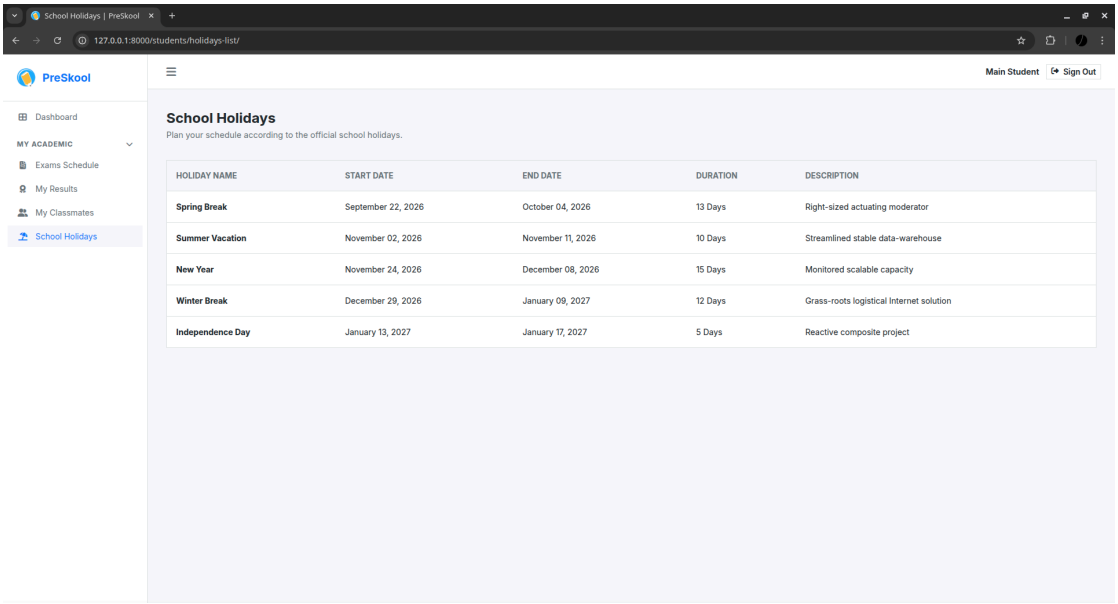
Exam Date
mm/dd/yyyy
Max Marks
100
Cancel
Add Exam


Student Dashboard | PreS
127.0.0.1:8000/students/dashboard/
PreSkool
Main Student
Sign Out
Dashboard
MY ACADEMIC
Exams Schedule
My Results
My Classmates
School Holidays
Dashboard
Welcome back, Main! Class: Grade 8
OVERALL AVERAGE
63.7%
NEXT EXAM
None scheduled
UPCOMING BREAK
Spring Break
Starts Sep 22
Recent Exam Results
View All
EXAM
SUBJECT
SCORE
Grade 8 - Final Exam
English
76 / 100
Grade 8 - Monthly Quiz
History
32 / 50
Grade 8 - Final Exam
Biology
72 / 100
Academic Info
Current enrollment status:
Grade 8
Admission No:
ADM10001
Student ID:
#1


Exams - PreSkool
127.0.0.1:8000/exams/
PreSkool
Main Student
Sign Out
Dashboard
MY ACADEMIC
Exams Schedule
My Results
My Classmates
School Holidays
Exams Schedule
Your upcoming examination schedule for Grade 8.
EXAM NAME
SUBJECT
DATE
MAX MARKS
ACTIONS
Grade 8 - Final Exam
ENGLISH
Mar 22, 2026
100
View Only
Grade 8 - Monthly Quiz
ENGLISH
Mar 21, 2026
50
View Only
Grade 8 - Monthly Quiz
ALGEBRA
Mar 20, 2026
100
View Only
Grade 8 - Monthly Quiz
BIOLOGY
Mar 20, 2026
50
View Only
Grade 8 - Monthly Quiz
HISTORY
Mar 19, 2026
50
View Only
Grade 8 - Final Exam
BIOLOGY
Feb 25, 2026
100
View Only
Grade 8 - Monthly Quiz
FRENCH
Feb 22, 2026
100
View Only
Grade 8 - Mid-Term Exam
ENGLISH
Feb 21, 2026
50
View Only
Grade 8 - Final Exam
ALGEBRA
Feb 17, 2026
100
View Only
Grade 8 - Final Exam
FRENCH
Feb 14, 2026
100
View Only
Grade 8 - Mid-Term Exam
HISTORY
Feb 11, 2026
100
View Only
Grade 8 - Final Exam
GEOMETRY
Feb 10, 2026
100
View Only


My Results | PreSkool
127.0.0.1:8000/students/results/
PreSkool
Main Student
Sign Out
Dashboard
MY ACADEMIC
Exams Schedule
My Results
My Classmates
School Holidays
My Exam Results
A full list of all your exam results and grades.
EXAM NAME | SUBJECT | DATE | MARKS OBTAINED | MAX MARKS | PERCENTAGE | STATUS
Grade 8 - Final Exam | English | Mar 22, 2026 | 76.09 | 100 | 76.1% | PASS
Grade 8 - Monthly Quiz | History | Mar 19, 2026 | 31.63 | 50 | 63.3% | PASS
Grade 8 - Final Exam | Biology | Feb 25, 2026 | 72.30 | 100 | 72.3% | PASS
Grade 8 - Monthly Quiz | French | Feb 22, 2026 | 77.10 | 100 | 77.1% | PASS
Grade 8 - Mid-Term Exam | English | Feb 21, 2026 | 33.19 | 50 | 66.4% | PASS
Grade 8 - Final Exam | Algebra | Feb 17, 2026 | 72.71 | 100 | 72.7% | PASS
Grade 8 - Final Exam | French | Feb 14, 2026 | 78.64 | 100 | 78.6% | PASS
Grade 8 - Mid-Term Exam | History | Feb 11, 2026 | 49.11 | 100 | 49.1% | FAIL
Grade 8 - Final Exam | Geometry | Feb 10, 2026 | 94.10 | 100 | 94.1% | PASS
Grade 8 - Mid-Term Exam | French | Feb 06, 2026 | 79.33 | 100 | 79.3% | PASS
Grade 8 - Mid-Term Exam | Geometry | Feb 06, 2026 | 86.52 | 100 | 86.5% | PASS
Grade 8 - Mid-Term Exam | Biology | Feb 04, 2026 | 94.09 | 100 | 94.1% | PASS


My Classmates - PreSkool
127.0.0.1:8000/students/my-class/
PreSkool
Main Student
Sign Out
Dashboard
MY ACADEMIC
Exams Schedule
My Results
My Classmates
School Holidays
My Classmates
Your peers in Grade 8
ADM. NO | STUDENT NAME | EMAIL
ADM36025 | Tiffany Jones | garciakimberly@example.com
ADM67498 | Angel Strickland | barbaracox@example.org
ADM46137 | Wesley Beltran | thomas95@example.org
ADM38160 | Cathy Warren | garyatkins@example.com
ADM73802 | Jennifer Brady | heathsabrina@example.org
ADM32531 | Mark Gonzalez | hawkinsvalerie@example.com
ADM8102 | Elaine Morris | smithmichael@example.net
ADM20187 | Allison Schroeder | zkaiser@example.com
ADM80977 | Joshua Rosales | valeriemartinez@example.com
ADM55386 | Austin York | james99@example.org
ADM23844 | Kirsten Browning | maydiane@example.org
ADM35525 | Charles Miller | anthonythomas@example.org


School Holidays | PreSkool
127.0.0.1:8000/students/holidays-list/
PreSkool
Main Student
Sign Out
Dashboard
MY ACADEMIC
Exams Schedule
My Results
My Classmates
School Holidays
School Holidays
Plan your schedule according to the official school holidays.
HOLIDAY NAME | START DATE | END DATE | DURATION | DESCRIPTION
Spring Break | September 22, 2026 | October 04, 2026 | 13 Days | Right-sized actuating moderator
Summer Vacation | November 02, 2026 | November 11, 2026 | 10 Days | Streamlined stable data-warehouse
New Year | November 24, 2026 | December 08, 2026 | 15 Days | Monitored scalable capacity
Winter Break | December 29, 2026 | January 09, 2027 | 12 Days | Grass-roots logistical Internet solution
Independence Day | January 13, 2027 | January 17, 2027 | 5 Days | Reactive composite project